

MAREVLO · PRODUCTION RAG

Optimization

Making it better — on purpose, in the right order, one measured change at a time.

PART II — MAKING IT WORK

Module 9

Production RAG: Building Retrieval-Augmented Generation Systems That Survive Real Users

What this module is about

In the last module we gave ourselves two things we had been missing: *eyes*, in the form of traces that let us see inside a single request, and a *scorecard*, in the form of an evaluation set that tells us honestly whether the system is getting better. With those two things in hand, we can finally do something we could not safely do before: improve the system on purpose. That is what optimisation means, and it is the subject of this module.

But here is the idea that makes this module different from a simple bag of tricks. Optimisation is not mainly about knowing clever techniques — it is about applying them **in the right order, one at a time, and keeping only the ones that measurably help**. A team that knows every advanced technique but applies them at random, judging by feel, will do worse than a team that knows only a few and applies them in order, measuring each one. So this module teaches both: the techniques, yes, but more importantly the discipline that turns them into real progress instead of expensive thrashing.

THE WHOLE MODULE IN ONE SENTENCE

Improve retrieval by climbing a ladder of techniques from highest-leverage to lowest — chunking, then filtering, then query rewriting, then re-ranking, and only last the prompt — changing one thing at a time and keeping it only if your evaluation score goes up.

Where we are

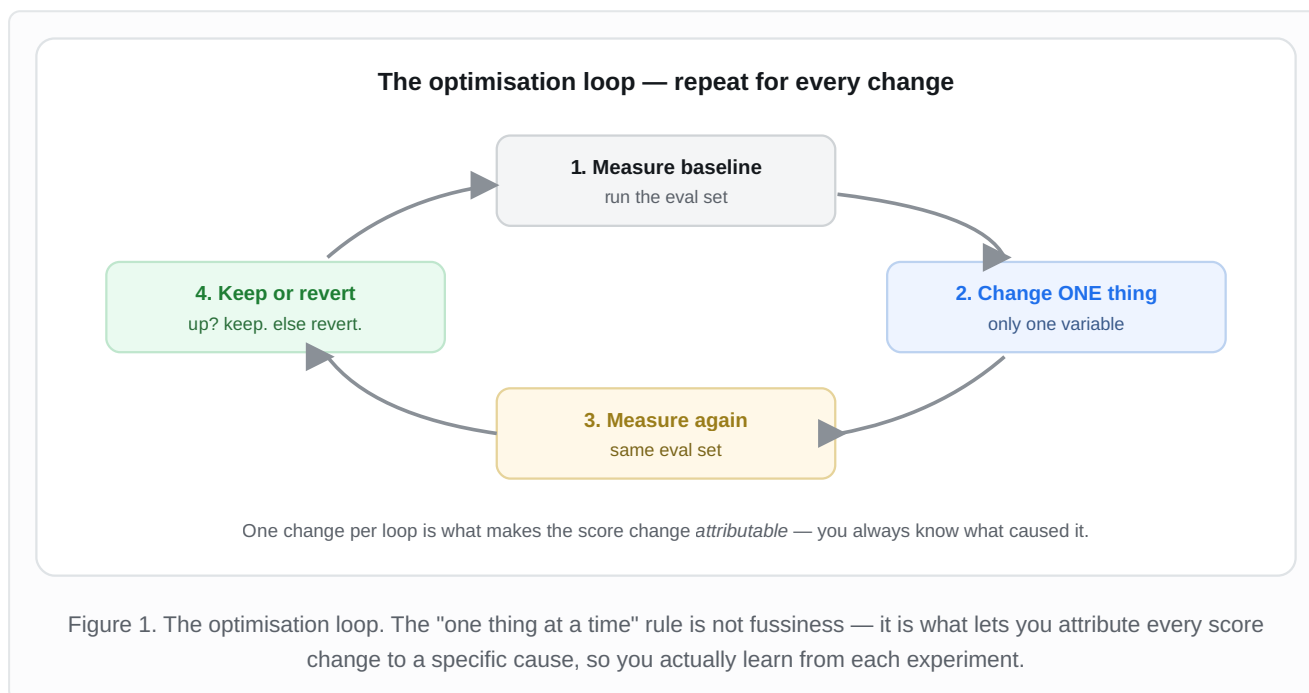
We have a working system (Module 5), better retrieval (Module 7), and the ability to see and measure it (Module 8). This module uses that measurement ability constantly: every technique we introduce is something you try, score against your evaluation set, and keep or discard. If you skipped straight here hoping for techniques alone, the most important sentence in the module is this — without the evaluation set from Module 8, none of what follows is optimisation; it is just guessing with extra steps.

SECTION 1

The optimisation loop: the discipline that makes it work

Before any technique, learn the loop. Every improvement follows the same four steps, and skipping them is how teams go sideways for months.

Every single change you make to improve the system should follow the same simple loop. First, you **measure your baseline** by running the evaluation set and recording the score. Second, you **change exactly one thing** — not three, one. Third, you **measure again** with the same evaluation set. Fourth, you **keep the change if the score went up, and revert it if the score went down or stayed flat**. Then you begin the loop again with the next change. It is unglamorous, and it is the entire difference between engineering and superstition.



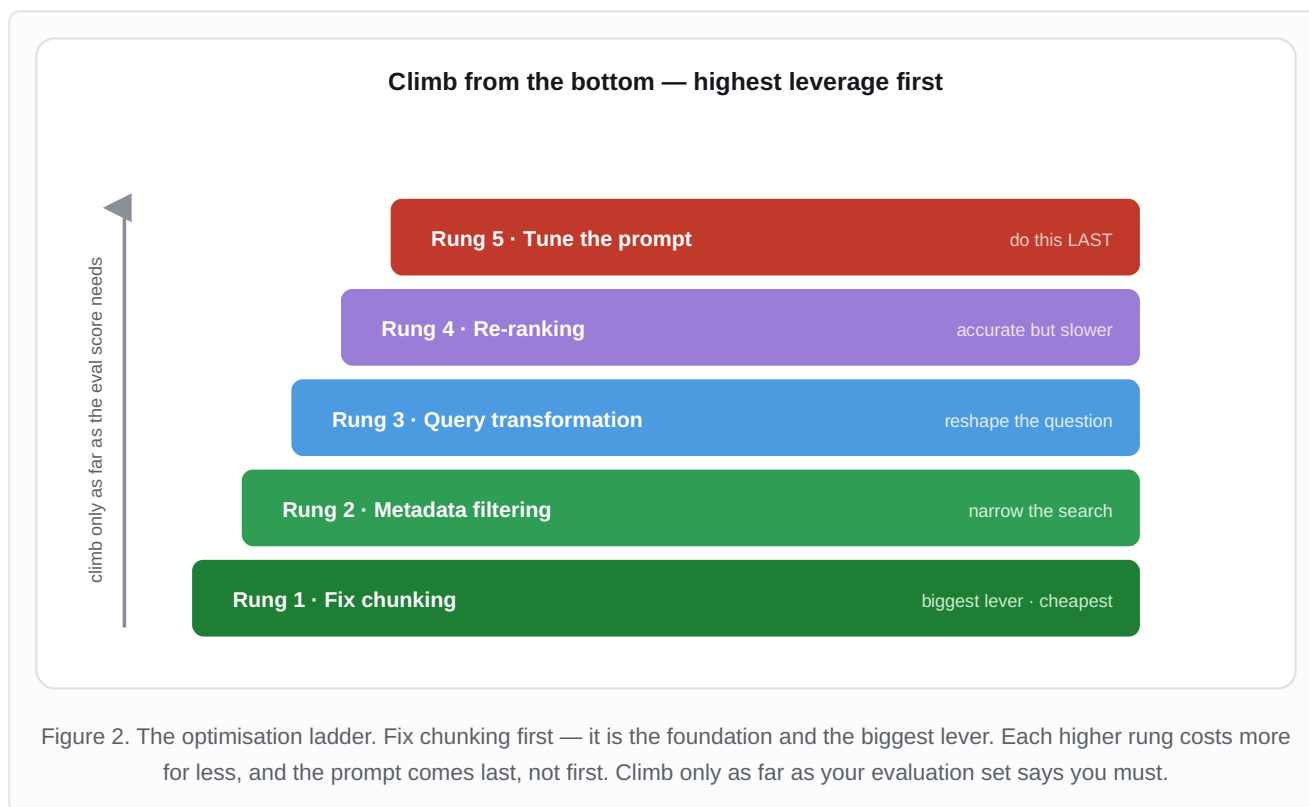
There is a second part of the discipline, just as important as the loop: **order matters**. Some improvements give you a lot of quality for very little effort, and some give you a little quality for a lot of effort and cost. The smart path is to do the cheap, high-impact things first, and only reach for the expensive, sophisticated ones if the cheap ones have not gotten you where you need to be. Reaching for an advanced technique while a basic one is still broken is one of the most common and most wasteful mistakes in the whole field.

SECTION 2

The optimisation ladder

Five rungs, ordered from highest leverage to lowest. Start at the bottom and climb only as far as your evaluation score requires.

Think of the available improvements as a ladder. The bottom rung is the foundation — the thing that gives you the most quality for the least effort, and the thing everything else stands on. As you climb, each rung gives a bit less leverage and costs a bit more. You start at the bottom and you climb only as high as you need to: if fixing the bottom rung gets your evaluation score where it needs to be, you may never need the higher rungs at all.



We will take the rungs in order. The first two — chunking and filtering — are techniques you already met in earlier modules; here we simply re-frame them as the first, highest-leverage optimisations and apply them with measurement. The last three — query transformation, re-ranking, and prompt tuning — get fuller treatment, because re-ranking in particular introduces a genuinely new and powerful idea.

RUNG 1 · HIGHEST LEVERAGE

Fix chunking — still the biggest lever

You met chunking in Module 2. Now you have an eval set, so you can finally tune it with evidence instead of guesswork.

It may feel anticlimactic that the most powerful optimisation is something from Module 2, but it is true: the way you cut your documents into pieces still affects answer quality more than any clever technique you can add on top. The difference now is that you have an evaluation set. In Module 2 you tuned chunking by eyeballing the pieces; now you can tune it by *measuring* — try a chunk size, run the eval set, note the score, try another size, compare. The eyeball test is still valuable, but the score makes it objective.

So before reaching for anything fancier, return to your chunking and test a few variations against your evaluation set: a smaller chunk size, a larger one, more overlap, structure-aware cutting for any tables or sections. Very often, a meaningful chunk of your quality problem dissolves right here, at the cheapest rung, and you never need to climb higher. Always check the foundation before adding floors on top of it.

RUNG 2

Metadata filtering — narrow before you search

You met filtering in Module 4. As an optimisation, it sharpens retrieval by removing whole categories of wrong answers.

The second rung is metadata filtering, which you met in Module 4. As an optimisation, its power is that it removes entire categories of wrong answers before the search even runs. If your evaluation set shows the system confidently returning last year's policy, or the wrong product's manual, the cause is almost always that the right meaning-based match existed but came from the wrong slice of your documents. Filtering by version, product, or date — applied *during* the search, as we stressed in Module 4 — fixes a whole class of these failures at once, cheaply. Test it the same way: add the filter, run the eval set, keep it if the score rises.

RUNG 3

Query transformation — reshape the question

Users phrase questions differently from how documents are written. Reshaping the question before retrieval closes that gap.

Now we reach the first genuinely new technique. So far we have searched using the user's question exactly as they typed it. But users phrase things in ways your documents do not: they use abbreviations the documents spell out, they make typos, they ask vaguely, or they simply word things differently from the source. **Query transformation** means improving the question *before* we search with it, so the search has a better chance of finding the right chunks.

There are a few flavours, increasing in power. The gentlest is **rewriting** — cleaning up the question, fixing typos, and turning a rambling sentence into a clear search query. Next is **expansion** — adding synonyms or spelling out abbreviations, so "what's the PTO policy?" also searches for "paid time off." The most powerful is **multi-query** — asking a model to generate several reworded versions of the question, searching with all of them, and merging the results. Multi-query casts a wider net: if any one phrasing matches the documents well, the right chunk is caught.

ANALOGY — ASKING A QUESTION SEVERAL WAYS

Imagine asking a librarian for help, and they ask the same question to three different colleagues, each in slightly different words, then pool everything the three of them found. If one colleague happened to phrase it in the way the archive is organised, the right document surfaces. Multi-query is doing exactly that — rephrasing your question several ways so that at least one phrasing lines up with how the documents are written.

LAB 9.1 — Multi-query: search several rewordings and merge (our own)

```
# We reuse `reciprocal_rank_fusion` from Module 7 to merge the results of
# several searches -- a nice example of earlier pieces composing.

def generate_query_variants(question, n=3):
    """Ask the model for n reworded versions of the question.

    We instruct it to keep the SAME meaning -- only the wording changes.
    Drifting from the original meaning is a real fault (see Faults section).
    """
    instruction = (
        f"Rewrite the question below in {n} different ways. "
        f"Keep the exact same meaning; only change the wording. "
        f"Return one rewording per line.\n\nQuestion: {question}"
    )
    response = model.invoke(instruction).content
    variants = [line.strip() for line in response.splitlines() if line.strip()]
    return [question] + variants  # always include the original too

def multi_query_search(question, k=4):
    """Search with several rewordings of the question, then merge."""
    rankings = []
    for variant in generate_query_variants(question):
        hits = store.similarity_search(variant, k=10)
        rankings.append([doc.page_content for doc in hits])

    # Merge all the rankings; chunks found by several wordings rise to the top.
    fused = reciprocal_rank_fusion(rankings)
    return fused[:k]
```

Query transformation costs an extra model call before retrieval, which adds a little time and money to every query, so as always you measure whether it earns its place. It tends to help most when your users are varied and informal, and to help least when they already phrase questions much like your documents.

RUNG 4 · THE POWERFUL ONE

Re-ranking — a second, more careful judge

Use fast search to gather a shortlist, then a slower, far more accurate model to pick the true best from it. The best of both worlds.

Re-ranking is the most powerful technique in this module, and it rests on understanding why our search so far has been fast but a little rough. To see it, you need one new distinction, and it is worth taking slowly because it explains the whole technique.

Everything we have done with embeddings uses what is called a **bi-encoder**. The name simply means the query and each document are turned into embeddings *separately*. The document embeddings can all be computed ahead of time and stored, so at query time we only embed the question and compare — which is why it is fast enough to search millions of chunks in an instant. But because the question and the document were each encoded on their own, never

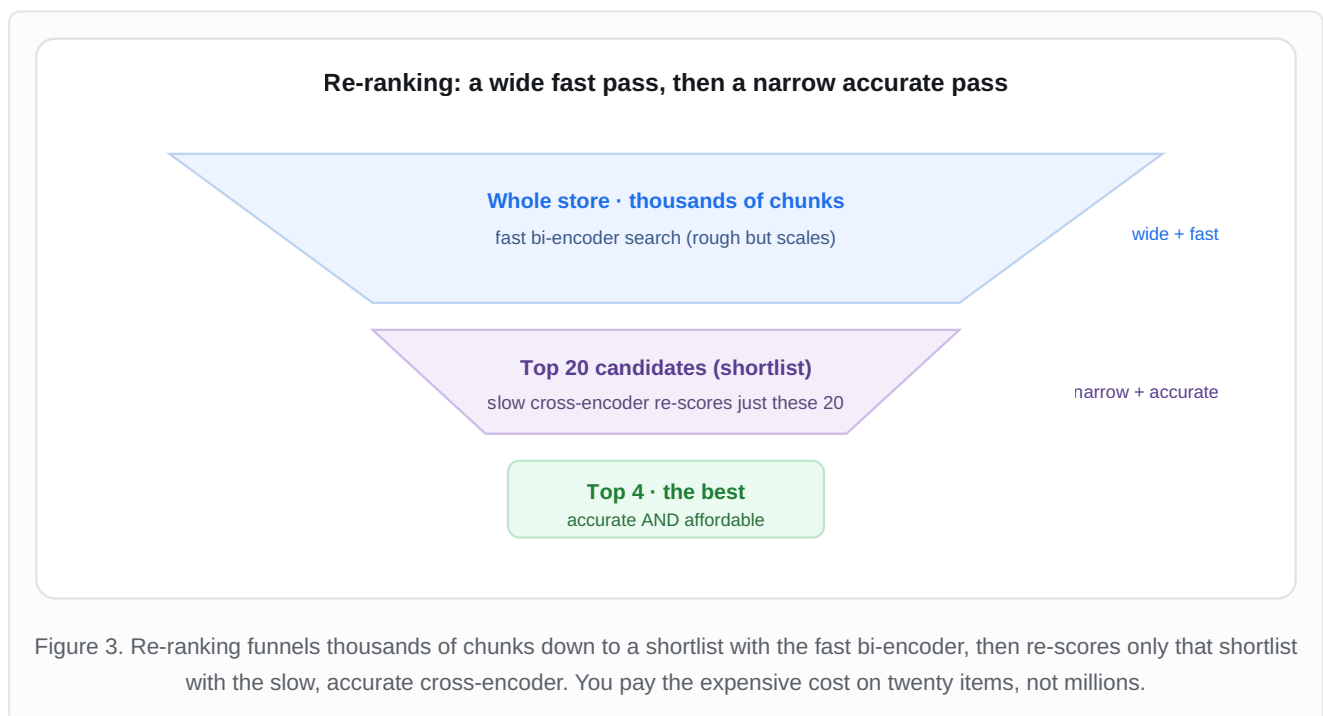
seen together, this method is a somewhat rough judge of whether a document truly answers a question. It is fast, but approximate.

A **cross-encoder** works the opposite way. It takes the question and one document *together*, as a single combined input, and outputs one number: how well this document answers this question. Because it considers them together, it is a far more accurate judge of relevance. The catch is speed: nothing can be pre-computed, because the score depends on the specific pairing, so you must run the model fresh for every question-document pair. Running it against millions of chunks for every query would be impossibly slow.

ANALOGY — PROFILE CARDS VERSUS A REAL CONVERSATION

A bi-encoder is like judging compatibility by reading two people's profile cards, each written independently, and seeing how well the cards match. You can scan thousands of cards quickly, but the match is rough. A cross-encoder is like having the two people actually talk for a few minutes — a far better judge of whether they get along, but you can only do a handful of conversations, not thousands. The smart move is obvious: use the quick profile cards to pick the twenty most promising matches out of thousands, then have real conversations with just those twenty.

That is exactly what re-ranking does. We use the fast bi-encoder search to gather a generous shortlist — say the top twenty candidates — from the whole store. Then we run the slow, accurate cross-encoder on just those twenty, re-scoring them carefully, and keep the true best four. We get the bi-encoder's scale *and* the cross-encoder's accuracy, paying the slow cost only on a small shortlist.



LAB 9.2 — Over-retrieve, then re-rank the shortlist (our own)

```
# A cross-encoder model that scores (question, document) pairs.
# Verify the import and model name against your installed version.
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def retrieve_and_rerank(question, shortlist=20, keep=4):
    """Fast pass to get a shortlist, then accurate pass to pick the best.

    shortlist: how many candidates the fast search gathers.
    keep:      how many we hand to the model after re-ranking.
    """
    # 1) Fast bi-encoder pass: gather a generous shortlist from the store.
    candidates = store.similarity_search(question, k=shortlist)

    # 2) Slow cross-encoder pass: score each (question, chunk) pair together.
    pairs = [(question, doc.page_content) for doc in candidates]
    scores = reranker.predict(pairs) # one accurate score per pair

    # 3) Sort the shortlist by the accurate score and keep the true best.
    ranked = sorted(zip(scores, candidates), key=lambda x: x[0], reverse=True)
    return [doc for score, doc in ranked[:keep]] # trim to the best `keep`
```

WHY RE-RANKING IS OFTEN THE HIGHEST-QUALITY SINGLE IMPROVEMENT

Re-ranking directly fixes two failures from earlier in the course at once: it rescues the right chunk that ranked just outside your top few (the rough bi-encoder had it at position seven; the cross-encoder promotes it), and it pushes out near-misses that only looked relevant. For many systems it is the single change that most improves answer quality. The cost is added time per query, so you weigh that against the quality gain — measured, of course, on the evaluation set.

MEDIUM CONFIDENCE — THE SIZE OF THE GAIN DEPENDS ON YOUR DATA

RUNG 5 · LAST, NOT FIRST

Tune the prompt — only after retrieval is right

The prompt matters enormously, but tuning it is the last rung, because it cannot fix information that was never retrieved.

It may surprise you that prompt tuning is the last rung, given how much we emphasised the prompt in Module 5. The prompt is indeed crucial — but tuning it is a *late* optimisation, for a precise reason: the prompt can only work with the chunks it is given. If the right chunk was never retrieved, no rewording of the prompt can conjure the missing information. So polishing the prompt while retrieval is still broken is effort spent on the wrong layer — the most common misdirected effort in all of RAG. Once the lower rungs have made retrieval reliable, *then* careful prompt wording earns real gains: clearer instructions, better handling of conflicting chunks, sharper formatting. Tune it last, when it is the thing actually holding you back, and measure each wording against the evaluation set like everything else.

FAULTS AND CORRECTIONS

The optimisation mistakes that waste the most effort

For each: the broken habit, why it wastes effort, and the fix.

FAULT 1 — Reaching for fancy techniques before fixing the basics

THE BROKEN HABIT

```
# Quality is poor, so we immediately add re-ranking and multi-query --  
# while the chunking is still cutting tables in half.  
answer = generate(question, retrieve_and_rerank(multi_query_search(question)))
```

WHY IT WASTES EFFORT

Advanced techniques add cost and complexity, and they cannot rescue a broken foundation. If your chunking is severing tables or producing blurry oversized pieces, re-ranking just more accurately ranks bad chunks, and multi-query just searches a broken index several ways. You pay for sophistication and get little, because the real problem was on the cheapest rung the whole time.

THE CORRECTION

```
# Climb the ladder in order. Fix chunking first and MEASURE before  
# adding anything fancier.  
baseline = run_eval(eval_set)  
improve_chunking()           # Rung 1 first  
if run_eval(eval_set) > baseline:  
    pass    # keep it, then consider the next rung only if still needed
```

FAULT 2 — Running the cross-encoder over the whole store

THE BROKEN APPROACH

```
# Trying to re-rank every chunk in the store -- the slow model on
# millions of pairs. Impossibly slow; the query never returns in time.
pairs = [(question, c) for c in every_chunk_in_store] # millions of pairs
scores = reranker.predict(pairs)
```

WHY IT WASTES EFFORT

The cross-encoder is accurate precisely because it cannot pre-compute anything, which is also why it is slow. Pointing it at the entire store means running a slow model millions of times per query — the request will time out or cost a fortune. The whole design of re-ranking exists to avoid this: the slow model must only ever see a small shortlist.

THE CORRECTION

```
# Fast pass first to get ~20 candidates, THEN the cross-encoder on those.
candidates = store.similarity_search(question, k=20) # fast, narrows the field
pairs = [(question, c.page_content) for c in candidates]
scores = reranker.predict(pairs) # slow model, but only 20 pairs
```

FAULT 3 — Over-retrieving for re-ranking but never trimming back

THE BROKEN APPROACH

```
# We fetch 20 candidates to re-rank... then send all 20 to the model.
candidates = store.similarity_search(question, k=20)
ranked = rerank(question, candidates)
prompt = build_prompt(question, ranked) # still 20 chunks -> noise and cost
```

WHY IT WASTES EFFORT

The point of fetching twenty is to give the cross-encoder a wide field to choose from — not to send all twenty to the model. If you forget to trim back to the best few after re-ranking, you have reintroduced the very problem of Module 7: too much context, which adds noise, slows the answer, and costs more. Re-ranking should make the final context *smaller and better*, not larger.

THE CORRECTION

```
# Re-rank the 20, then keep only the true best few for the prompt.
ranked = rerank(question, candidates)
best = ranked[:4] # trim to the best
prompt = build_prompt(question, best)
```

FAULT 4 — Query rewriting that drifts from the original meaning

THE BROKEN APPROACH

```
# Asking the model to "improve" the question with no constraint, so it
# quietly changes what was actually asked.
better = model.invoke(f"Make this question better: {question}").content
```

WHY IT WASTES EFFORT

If you let the model freely "improve" a question, it may broaden it, narrow it, or answer a slightly different question than the user asked. You then retrieve chunks for the wrong question and confidently answer something the user never wanted. Query transformation must change the *wording* while preserving the *meaning*; an unconstrained rewrite breaks that contract.

THE CORRECTION

```
# Constrain the rewrite to preserve meaning, and keep the original too.
instruction = (f"Reword this question, keeping the EXACT same meaning. "
              f"Do not add or remove any requirement.\n\n{question}")
variants = [question, model.invoke(instruction).content] # original + faithful reword
```

FAULT 5 — Tuning the prompt first, before retrieval is fixed

THE BROKEN HABIT

```
# Answers are wrong, so we spend days rewording the prompt -- while the
# right chunk is never being retrieved in the first place.
for wording in many_prompt_versions:
    test(wording) # none of them can fix missing information
```

WHY IT WASTES EFFORT

The prompt can only use the chunks it is handed. If retrieval is failing to surface the right chunk, every prompt wording in the world will still be working from the wrong information, so you spend days polishing a layer that was never the problem. This is the single most common misdirected effort in RAG — fixing generation when the failure is in retrieval.

THE CORRECTION

```
# First confirm retrieval surfaces the right chunk (check the trace from
# Module 8). Only tune the prompt once the right chunk is reliably there.
trace = answer_with_trace(question)
# Is the needed chunk in trace["retrieved"]? If not, fix retrieval first.
```

FAULT 6 — Chasing the eval score while ignoring latency and cost

THE BROKEN HABIT

```
# A change raises the eval score by 1%, but triples the response time  
# and doubles the cost. We keep it because "the score went up."  
if run_eval(eval_set) > baseline:  
    keep_change() # ignoring that it now takes 3x as long
```

WHY IT WASTES EFFORT

Answer quality is not the only thing that matters in production. A change that nudges the score up but makes every answer far slower or much more expensive can be a net loss — users feel the delay, and the bill grows. Optimising one number in isolation can quietly harm the things that number does not capture. A real decision weighs quality against speed and cost together.

THE CORRECTION

```
# Track quality AND latency AND cost for each change, and decide together.  
score = run_eval(eval_set)  
p95   = percentile(latencies, 95) # from Module 8  
cost  = average_cost_per_query()  
# Keep the change only if the quality gain justifies its latency/cost.
```

Checkpoint: try it yourself

This exercise has you climb the ladder with measurement, and feel the power of re-ranking directly. It takes about forty minutes and is the most complete optimisation practice in the course.

1. Run your evaluation set from Module 8 and record the baseline score. Write it down — this is the number every change will be compared against.
2. Start at Rung 1: try two different chunk sizes, running the eval set after each. Keep whichever scores best, and notice how much (or how little) of your quality problem this cheapest rung solved.
3. Now add re-ranking from Lab 9.2: over-retrieve twenty candidates, re-rank them, and keep the best four. Run the eval set again. For most systems you will see a clear jump — that is the cross-encoder rescuing good chunks the fast search ranked too low.
4. Finally, measure the cost of that re-ranking jump: time a few queries with and without it. Now you can make the real production decision — was the quality gain worth the extra time? That judgement, made with numbers, is exactly what optimisation is.

WHAT YOU SHOULD NOTICE

Two things. First, the cheapest rung (chunking) often does more than you expected, which is why we always start there. Second, when you added re-ranking, you did not hope it helped — you saw the score move and you saw the time it cost, and you decided with evidence. That combination of climbing in order and deciding with numbers is the whole discipline of this module.

What to carry into Module 10

THE SIX THINGS TO REMEMBER

One, optimisation is a loop: measure the baseline, change one thing, measure again, keep or revert — one change at a time, so every result is attributable. **Two**, climb the ladder in order of leverage, starting with the cheapest, highest-impact rung. **Three**, chunking is still the biggest lever, and now you can tune it by measuring, not just by eye. **Four**, query transformation reshapes the question to match how documents are written, and multi-query casts a wider net. **Five**, re-ranking pairs a fast rough search with a slow accurate one — gather a shortlist with the bi-encoder, pick the true best with the cross-encoder — and is often the single biggest quality gain. **Six**, tune the prompt last, because it cannot fix information that was never retrieved, and always weigh quality against the latency and cost a change adds.

We have made the system as good as it can be at small scale. But "good" and "good at scale" are different questions. What happens to quality, speed, and cost when the store holds millions of chunks and hundreds of users arrive at once? In Module 10 we face the real costs of running RAG at scale — and several of the trade-offs we just measured will look very different when the numbers get large.